

Computer Science & IT

Database Management System



Transactions & concurrency control

Lecture No. 09

By- Vishal Sir



Recap of Previous Lecture



Topic

Concurrency control protocols



Topic

Simple use of shared and exclusive locks



Topic

Two phase lock (2PL)





Topics to be Covered



✓ Topic

Basic 2PL and lock upgrading/downgrading

✓ Topic

Problems possible with Basic 2PL

✓ Topic

Strict 2PL

✓ Topic

Conservative 2PL

✓ Topic

Rigorous 2PL



Topic : Concurrency Control Protocols

- ✓ • Concurrency Control Protocols are responsible for avoiding the execution of non-serializable schedules.
- ✓ • Concurrency Control Protocols are suggested to ensure serializability



Topic : Concurrency Control Protocols

- There are two types of concurrency control protocols.

✓ 1. Lock based Protocols ✓

★ Two Phase Locking Protocol (2PL)

- ✓ (i) Basic 2PL
- ✓ (ii) Strict 2PL
- ✓ (iii) Conservative 2PL
- ✓ (iv) Rigorous 2PL

✓ 2. Time-stamp based Protocols

- (i) Basic Time stamp ordering Protocol.
- (ii) Thomas write Time-stamp ordering Protocol



Topic : Lock Based Protocols

- Lock is a variable associated with a data item that describes status of data item with respect to possible operation that can be applied to it.

There are two types of locks used,

1. **Shared LOCK [S]**: It is read only lock.

If transaction wants to perform only read opⁿ on a dataitem, then transaction will request for a shared lock on that dataitem

2. **Exclusive LOCK [X]**: It is read as well as write lock.

T
~~R(A)~~
We are not allowed to perform read or write opⁿ directly

T
Requesting for shared lock on dataitem 'A'
~~S(A)~~ ← "Let granted"
R(A) ✓ {Allowed}
~~W(A)~~ not allowed, transaction T does not have write permission w.r.t. dataitem 'A'
~~R(B)~~ Not allowed, dataitem 'B' is not locked by transaction T.



Topic : Lock Based Protocols

- Lock is a variable associated with a data item that describes status of data item with respect to possible operation that can be applied to it.

There are two types of locks used,

1. **Shared LOCK [S]:** It is read only lock.

T
~~W(A)~~
Not allowed

2. **Exclusive LOCK [X]:** It is read as well as write lock.

{ Transaction can request for an Exclusive lock
On dataitem A Only if transaction wants to perform
at least one write opⁿ on dataitem 'A' }

Requesting an Exclusive lock on dataitem 'A'

T
X(A) ← let granted
R(A) ✓ (Allowed)
W(A) ✓ (Allowed)
S(B) ✓
R(B) ✓ (Allowed)
~~W(B)~~ Not allowed
~~R(C)~~ - / -
~~W(C)~~ - / -



Topic : Lock Compatibility Table

Locks held by Transaction T_i

data item 'A'	Locks held by Transaction T_i	
	Shared lock 'S'	Exclusive lock 'X'
Shared lock 'S'	(✓) Allowed	Not allowed
Exclusive lock 'X'	Not allowed	Not allowed

locks
requested by
transaction T_j

Note:-

To ensure serializability a non-serializable schedule must not be allowed to execute.



Topic : Simple use of shared and exclusive locks

ie, No restriction { ie, data items can be locked/unlocked }
in any order at any time

Consider the following Schedule.

	T ₁	T ₂
✓	X(A)	
✓	R(A)	
✓	W(A)	
Unlock(A) ≡	U(A)	S(A) ✓
		R(A) ✓
		S(B) ✓
		R(B) ✓
		U(A)
		U(B)
✓	X(B)	
✓	R(B)	
✓	W(B)	
	U(B)	

Given schedule
is a non-serializable
Schedule.

But it can be allowed
to execute using
Simple use of
Shared and Exclusive
lock.

A non-serializable
Schedule may be allowed
to execute,
∴ Simple use of Shared
and Exclusive locks
does not ensure
serializability.



Topic : Basic Two Phase Locking Protocol

→ There are two phases in Two phase locking protocol

- ✓ 1. Growing phase / Lock acquiring phase
- ✓ 2. Shrinking phase / Lock releasing phase

→ In Two Phase Locking Protocol, transaction T is allowed to request for a lock ^{on any data item} only if transaction T has not performed any unlock operation.

eg:

Transaction (T)

Lock acquiring
Phase

(or)

Growing Phase

S(A)
R(A)
X(B)
R(B)
X(C)
S(D)
W(B)
R(C)
R(A)

U(A)

R(D)

W(C)

U(C)

S(A)/X(A)

S(E)/X(E)

W(B)

U(B)

U(D)

Shrinking Phase
starts

Shrinking Phase
(or)
Lock releasing Phase

not allowed

not allowed

① Q. Check whether the schedule is allowed to execute using 2PL or not?

⑤

	T ₁	T ₂
1	X(A) R ₁ (A) W ₁ (A)	
		S(A) R ₂ (A)
		R ₂ (B)
	R ₁ (B) W ₁ (B)	

it will be denied because 'A' is already locked by T₁ in Exclusive mode

If a lock request is denied, then that transaction will go into a time-out period and it will request for the same lock once time-out period is over.

① Q. Check whether the schedule is allowed to execute using 2PL or not?

⑤

	T ₁	T ₂
✓	X(A) R ₁ (A) W ₁ (A) U(A)	S(A) ✓ R ₂ (A) S(B) R ₂ (B) U(A) U(B) -
	 X(B) ✓ R ₁ (B) W ₁ (B)	

denied
because T₁
has already
performed unlock
op

① ②

⑤

	T_1	T_2
✓	$X(A)$ $R_1(A)$ $W_1(A)$ $X(B)$ $U(A)$	$S(A)$ ✓ $R_2(A)$ $S(B)$ der $R_2(B)$
	$R_1(B)$ $W_1(B)$	

① Q. Check whether the schedule is allowed to execute using 2PL or not?

⑤

T_1	T_2
$R_1(A)$ $W_1(A)$	$R_2(A)$ $R_2(B)$
$R_1(B)$ $W_1(B)$	

→ Schedule is a non-serializable schedule and it is not allowed to execute using "2PL" protocols.

② Q. Check whether the schedule is allowed to execute using 2PL or not?

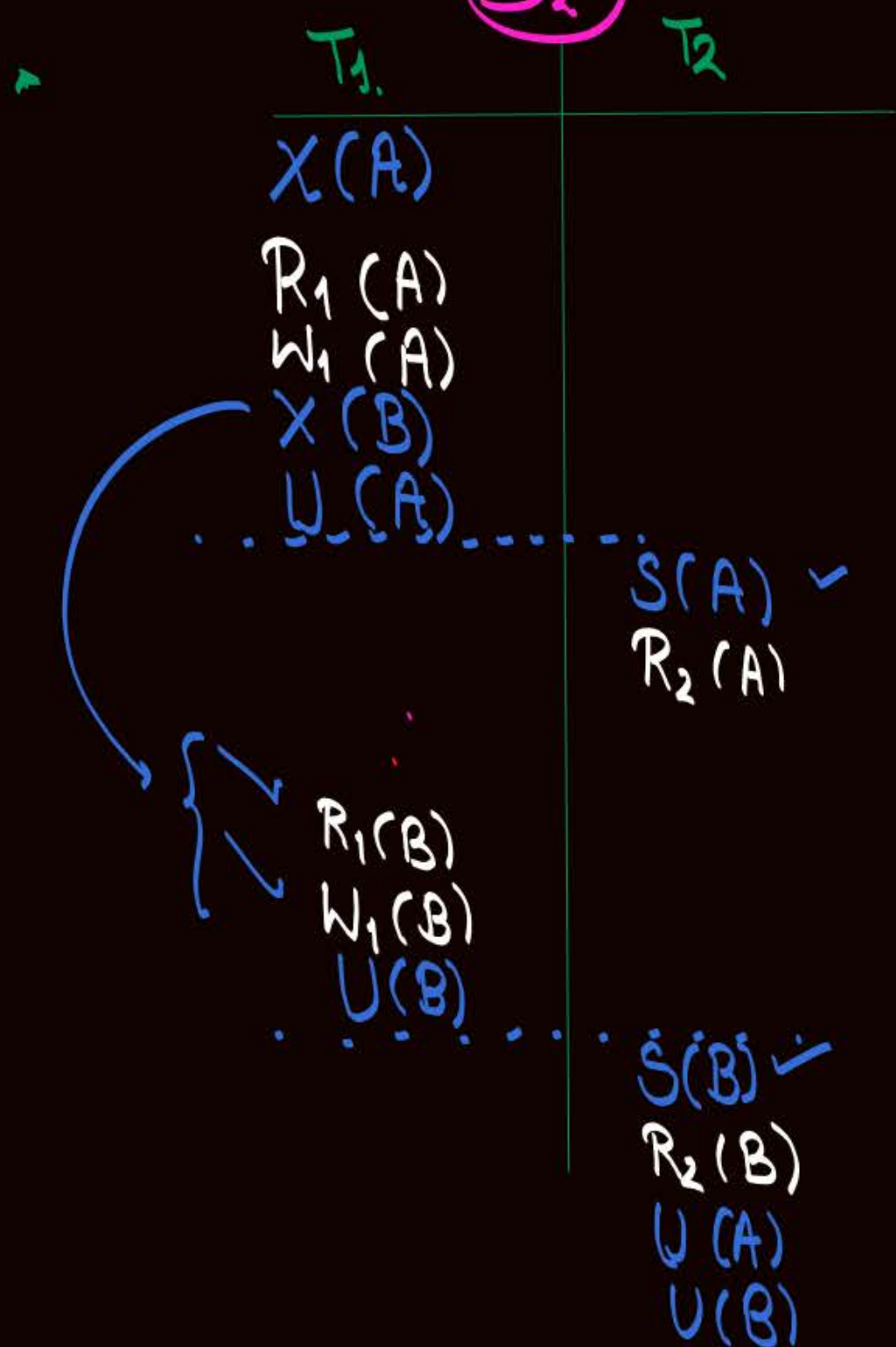
Precedence graph



Ayclic

∴ Schedule is a C.S.S.

It is a Conflict Serializable Schedule and it is allowed to execute using "2PL protocols"



③ Q. Check whether the schedule is allowed to execute using 2PL or not?

S₃

Until transaction T₂ performs all the operation on data item 'B', it will not unlock 'B'.

T ₁	T ₂
	X(B)
	R(B)

S(A)
R(A)

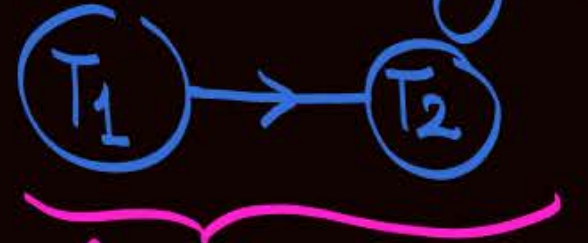
~~S(B)~~
R(B)

∴ this S(B) can never be granted at this position
∴ Not allowed by 2PL

W(B)

Schedule is a C.S.S., but it is not allowed by "2PL"

Precedence graph



Acyclic
∴ it is a C.S.S.

④ Q. Check whether the schedule is allowed to execute using 2PL or not?

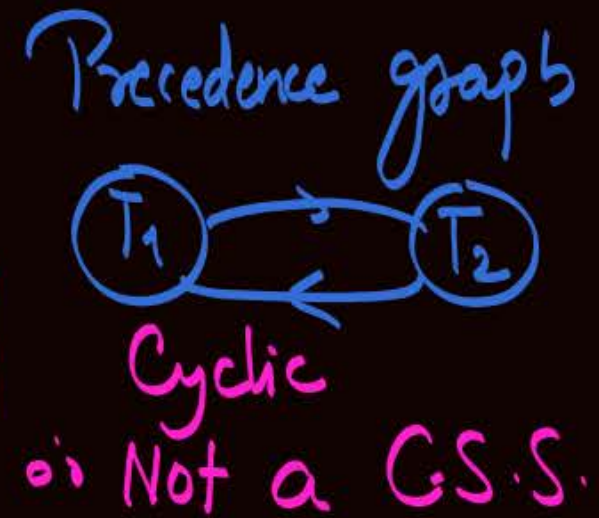
S₄

T ₁	T ₂
X(B)	
R(B)	
	X(B)
	W(B)
W(B)	
	W(B)

~~X(B)~~ denied
 ↳ can not be fulfilled
 ∴ Not allowed by 2PL

It is not a CSS;
 but it is a View
 Serializable Schedule

⇓
 It is not
 allowed by 2PL



But
 $S_4 \equiv T_1 \rightarrow T_2$
 ∴ S₄ is a
 Serializable
 Schedule
 (i.e. View
 Serializable)



Topic : Basic Two Phase Locking Protocol

1. 2PL ensures serializability, i.e., a non-serializable schedule is never allowed to execute using 2PL.
2. If schedule is allowed to execute using 2PL, then schedule is conflict serializable schedule, but every conflict serializable schedule need not be allowed to execute by 2PL.
 { We can also say 2PL ensures Conflict serializability }
 { i.e. a non-C.S.S. is never allowed by 2PL }
3. If schedule is not a conflict serializable schedule, then that schedule is not allowed to execute using 2PL.
4. If schedule is allowed to execute using 2PL, then schedule is conflict serializable schedule and conflict equivalent serial schedule can be given by order of lock points.

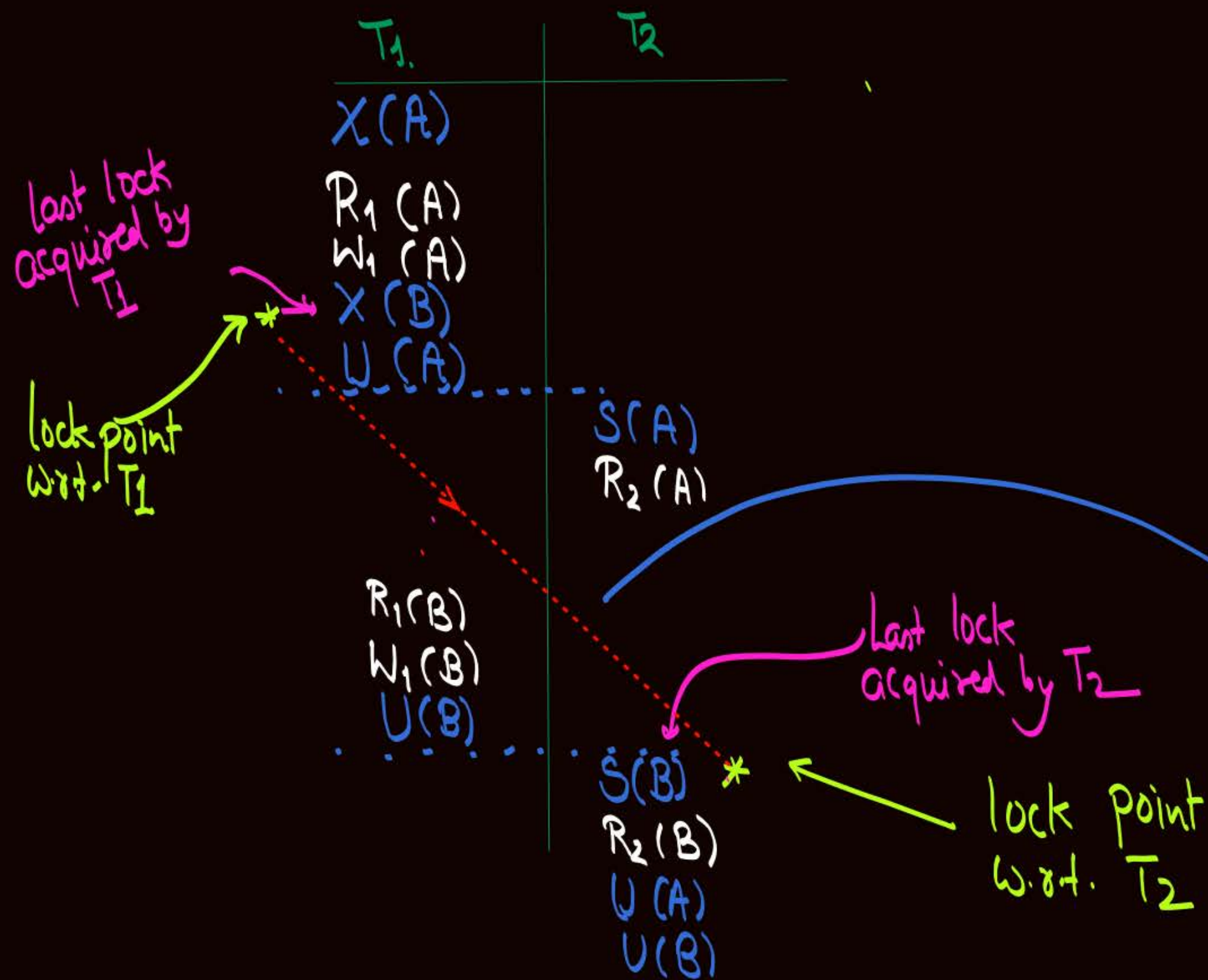


Topic : Lock Point



- The Point at which the growing phase ends, i.e., the point at which transaction takes the final lock it needs to carry on its work. is defined as lock point w.r.t. that transaction.

eg.



It is a Conflict serializable Schedule, and it is allowed to execute using "2PL protocols"

Conflict equivalent serial Schedule w.r.t. order of lock point is

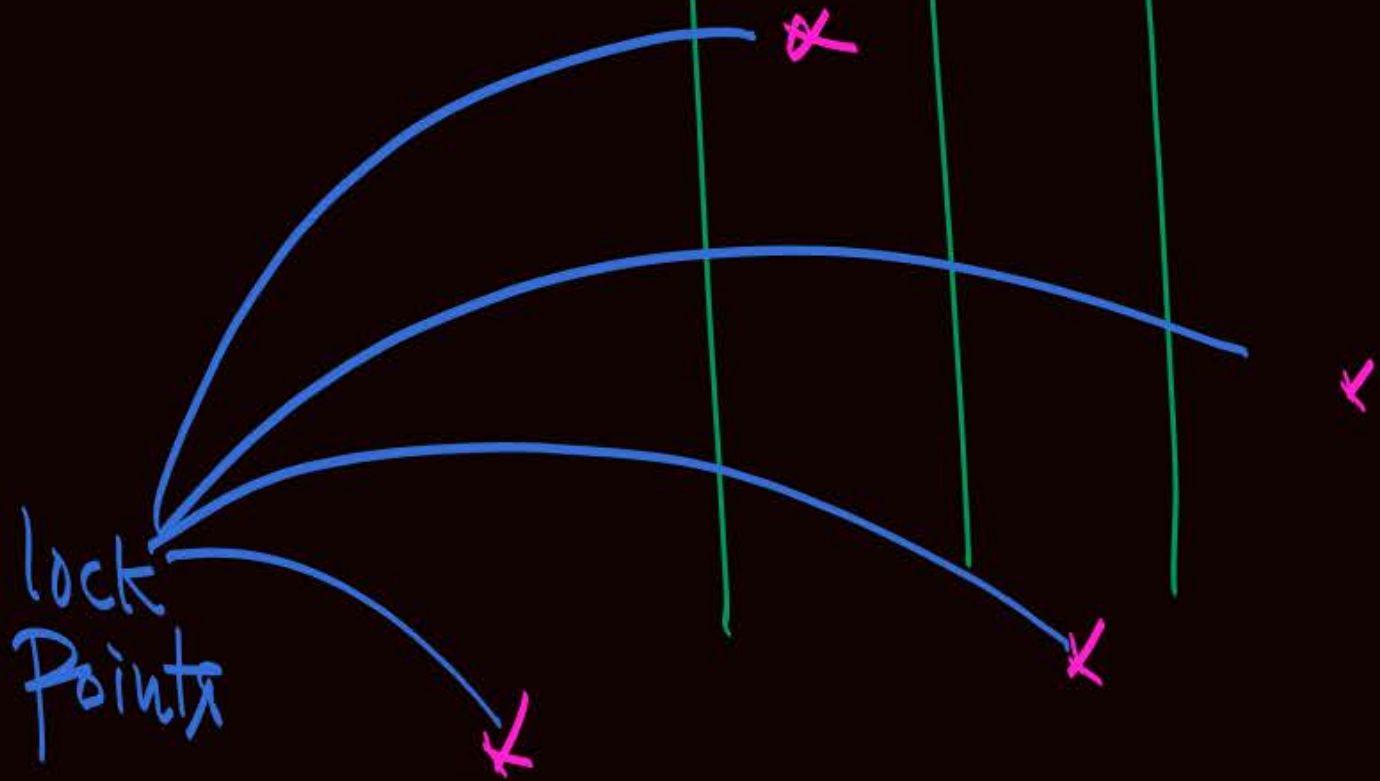
$T_1 \rightarrow T_2$

eg. Lock Points,

Consider the following

'S'

T ₁	T ₂	T ₃	T ₄
----------------	----------------	----------------	----------------



Schedule 'S', and assume that
Schedule 'S' is allowed by 2PL,

then conflict equivalent serial
Schedule based on order of
lock point is

→ T₂ → T₄ → T₃ → T₁



Topic : Basic 2PL with lock upgrading

- If lock conversion (upgrading/downgrading) is allowed, then we can upgrade shared lock into exclusive locks in the Growing Phase and we can downgrade from the exclusive lock to shared lock in the shrinking phase.

Upgrade: Shared to Exclusive
Downgrade: Exclusive to Shared

- We are not allowed to upgrade if transaction is already in the shrinking phase
- If transaction is not already in its shrinking phase, then we are allowed to downgrade, but as soon as we perform downgrade opn the shrinking phase of transaction will begin

Q. Check whether the schedule is allowed to execute using 2PL without lock upgrading or not?

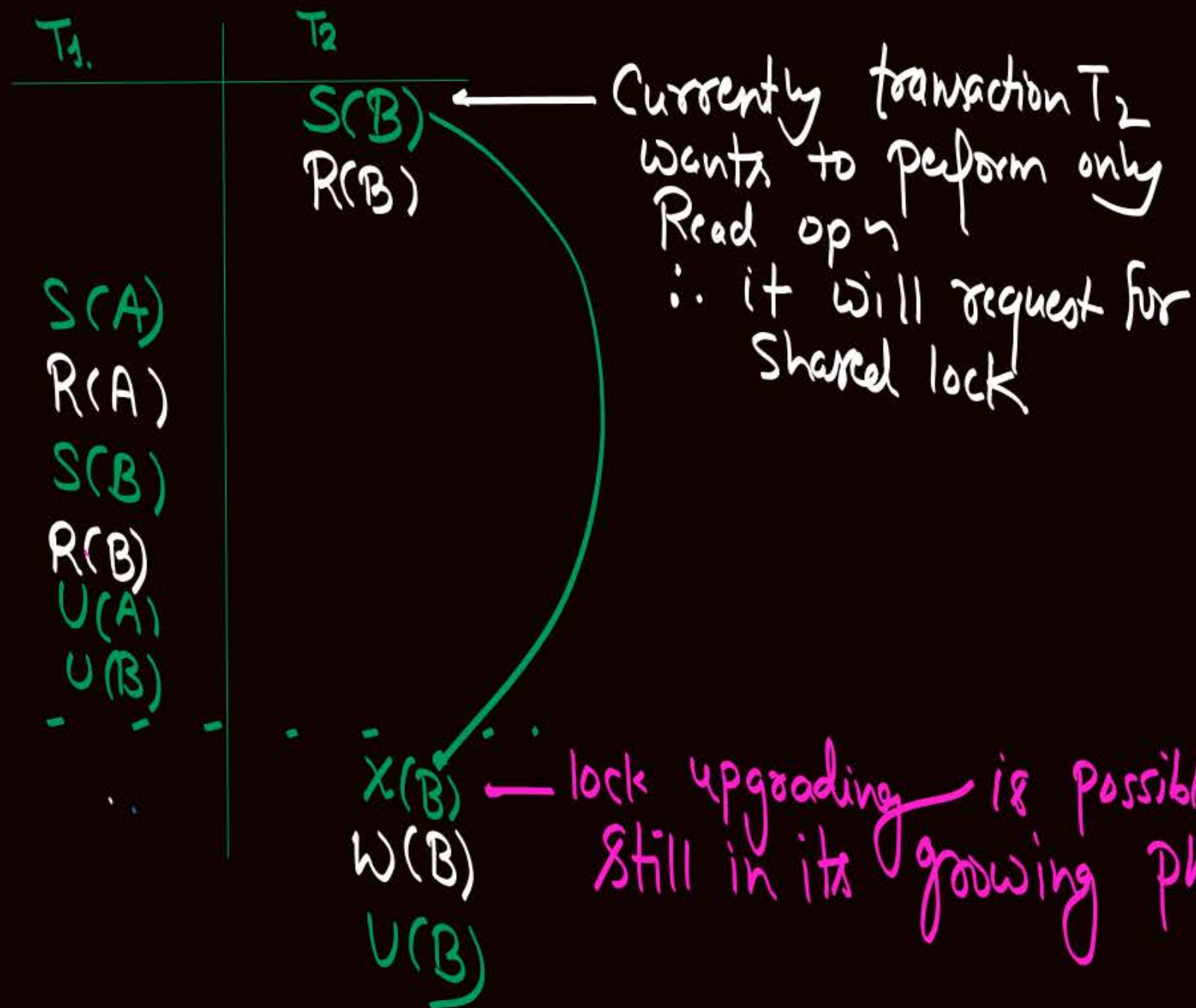
S₃

T ₁	T ₂
	X(B) R(B)
S(A) R(A) S(B) R(B)	
	W(B)

denied
∴ Not allowed

• Given schedule is C.S.S.
but not allowed by 2PL
if lock upgrading is not allowed

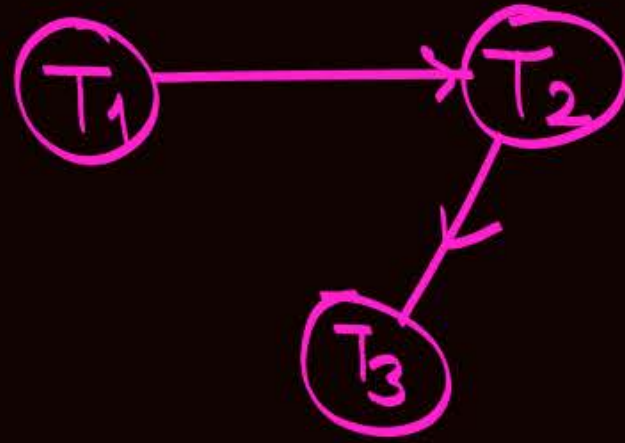
Q. Check whether the schedule is allowed to execute using 2PL with lock upgrading or not?



It is a C.S.S. and allowed to execute using 2PL if lock upgrading is allowed

Q: Consider the following schedule, Check whether the schedule is allowed to execute using 2PL without lock upgrading or not?

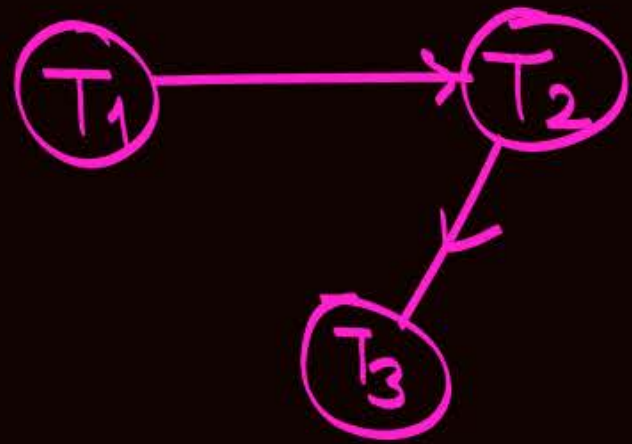
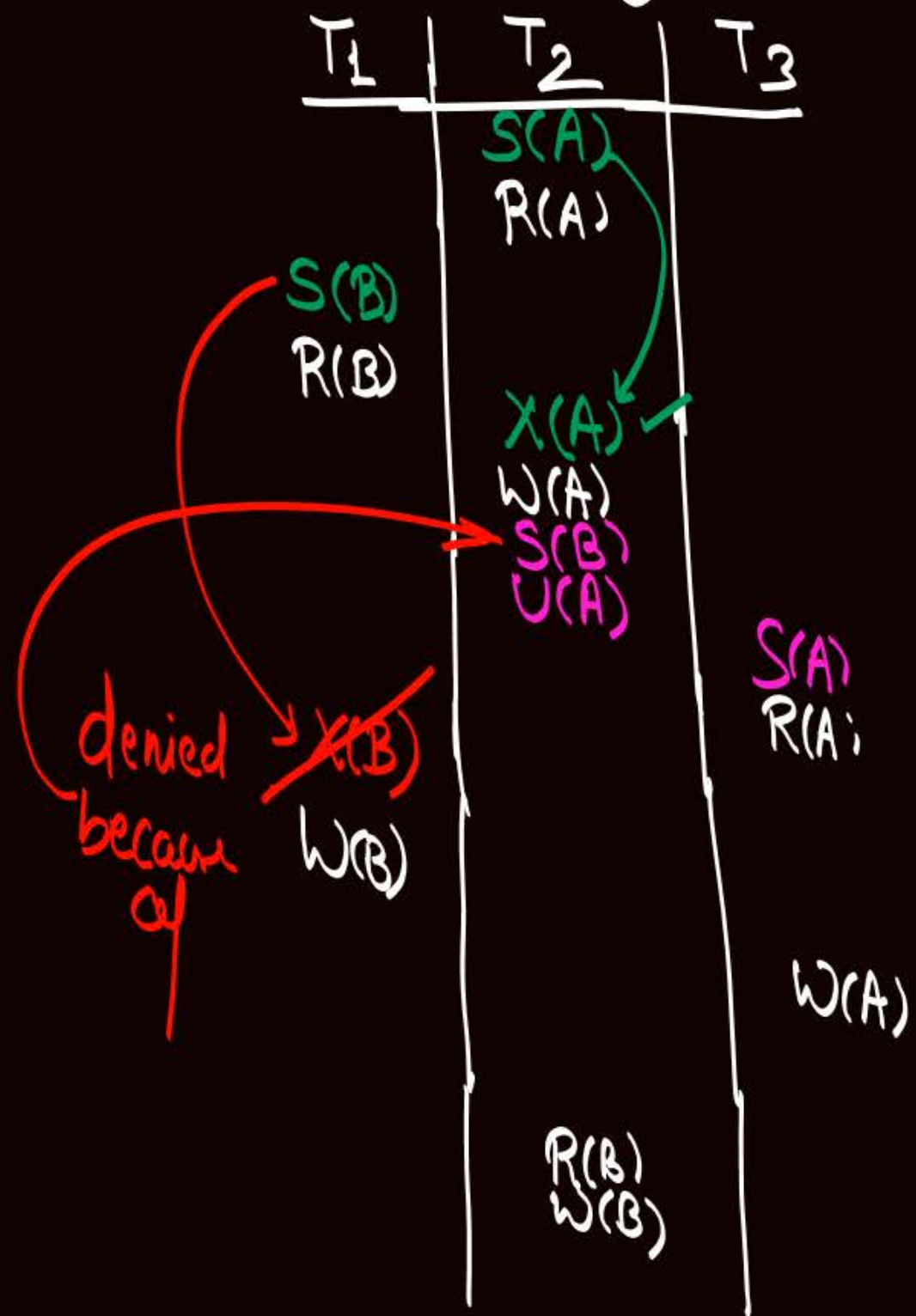
T ₁	T ₂	T ₃
	X(A) R(A)	
X(B) R(B)		
	W(A) X(B) denied	
		U(A) X
		X(A) X
		R(A)
W(B)		
		W(A)
	R(B) W(B)	



Acyclic
∴ Schedule is C.S.S.

but not allowed by 2PL
without lock upgrading

Q: Consider the following schedule, Check whether the schedule is allowed to execute using 2PL with lock upgrading or not?



Acyclic
 ∴ Schedule is C.S.S., But
 Not allowed by 2PL
 even when lock upgrading
 is allowed.

- Note:-
- ① If Lock upgrading is allowed, then some extra conflict serializable schedule may be allowed to execute using 2PL, but still there will be some conflict serializable schedule that are not allowed by 2PL even when lock upgrading is allowed.
 - ② If schedule is not a C.S.S., then it is never allowed by 2PL does not matter whether lock upgrading is allowed or not.



Topic : Problems possible with Basic 2PL

A schedule that is allowed to execute using basic 2PL protocol may suffer from,

- ① Irrecoverability { Can be solved using Strict-2PL }
- ② Deadlock { Can be solved using Conservative 2PL }
- ③ Starvation { No solution }



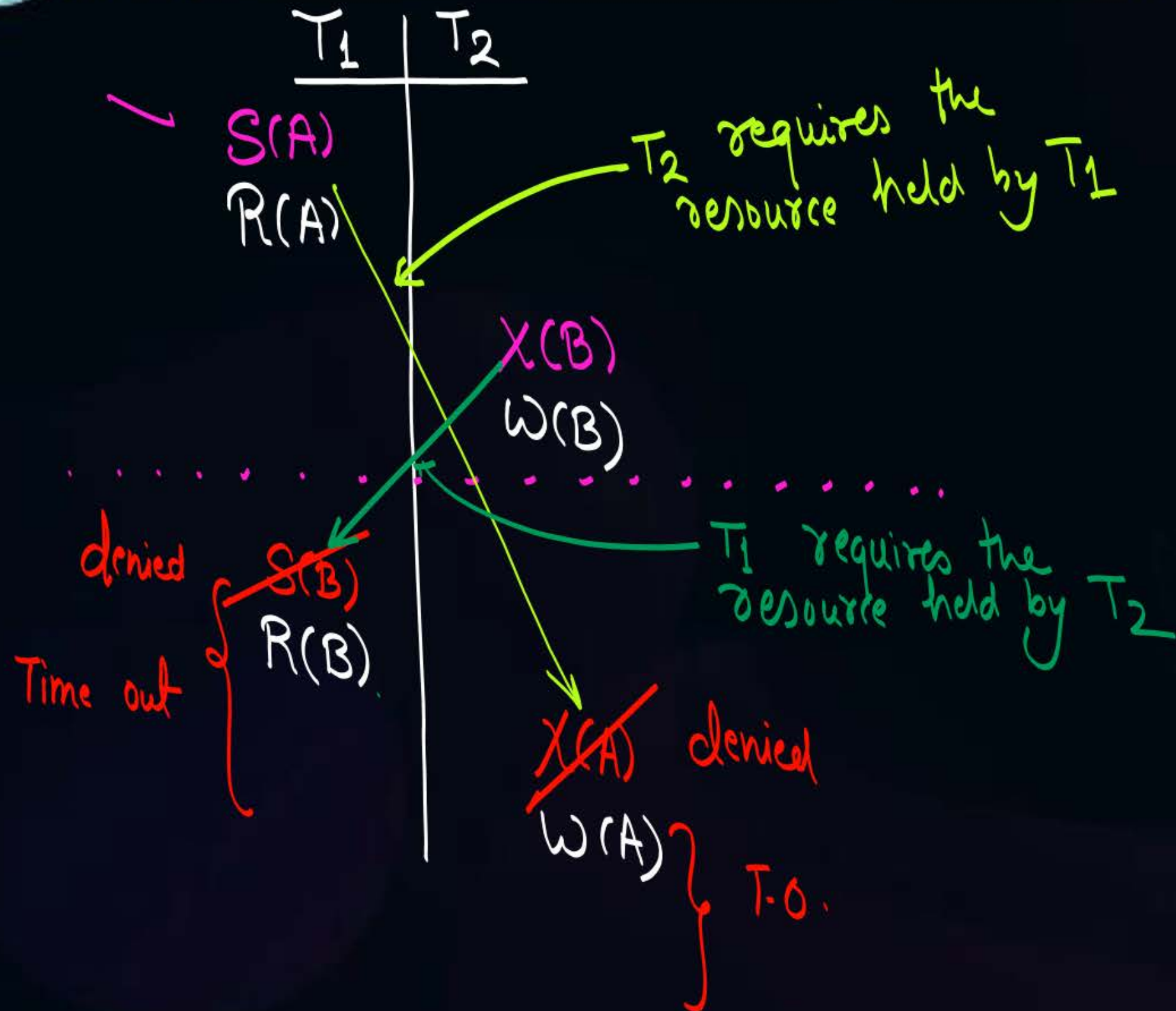
- * Schedule is allowed to execute using 2PL, but schedule is irrecoverable

lensafé

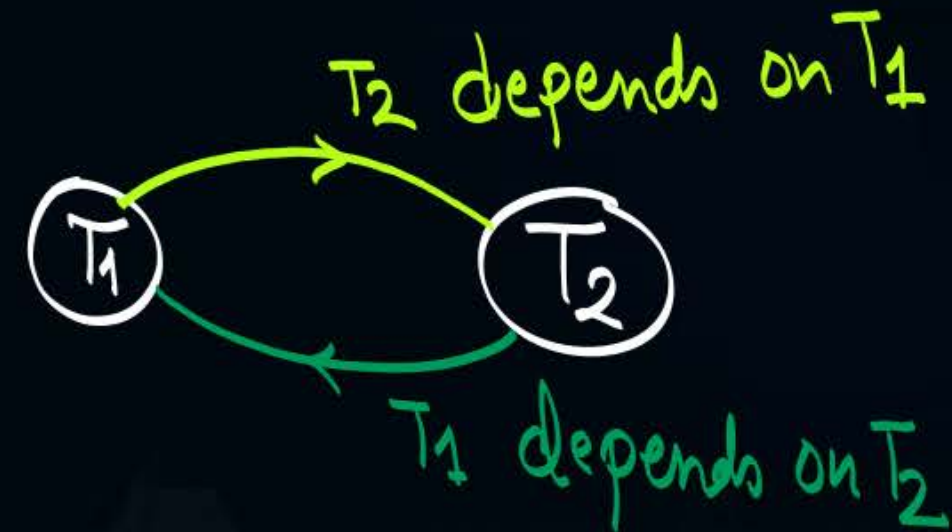
Dependent transaction
(i.e., T_2) committed
before the
Commit/Rollback of
the transaction on which
it depends (i.e., T_1)
∴ irrecoverable
Schedule



Topic : Deadlock with Basic 2PL



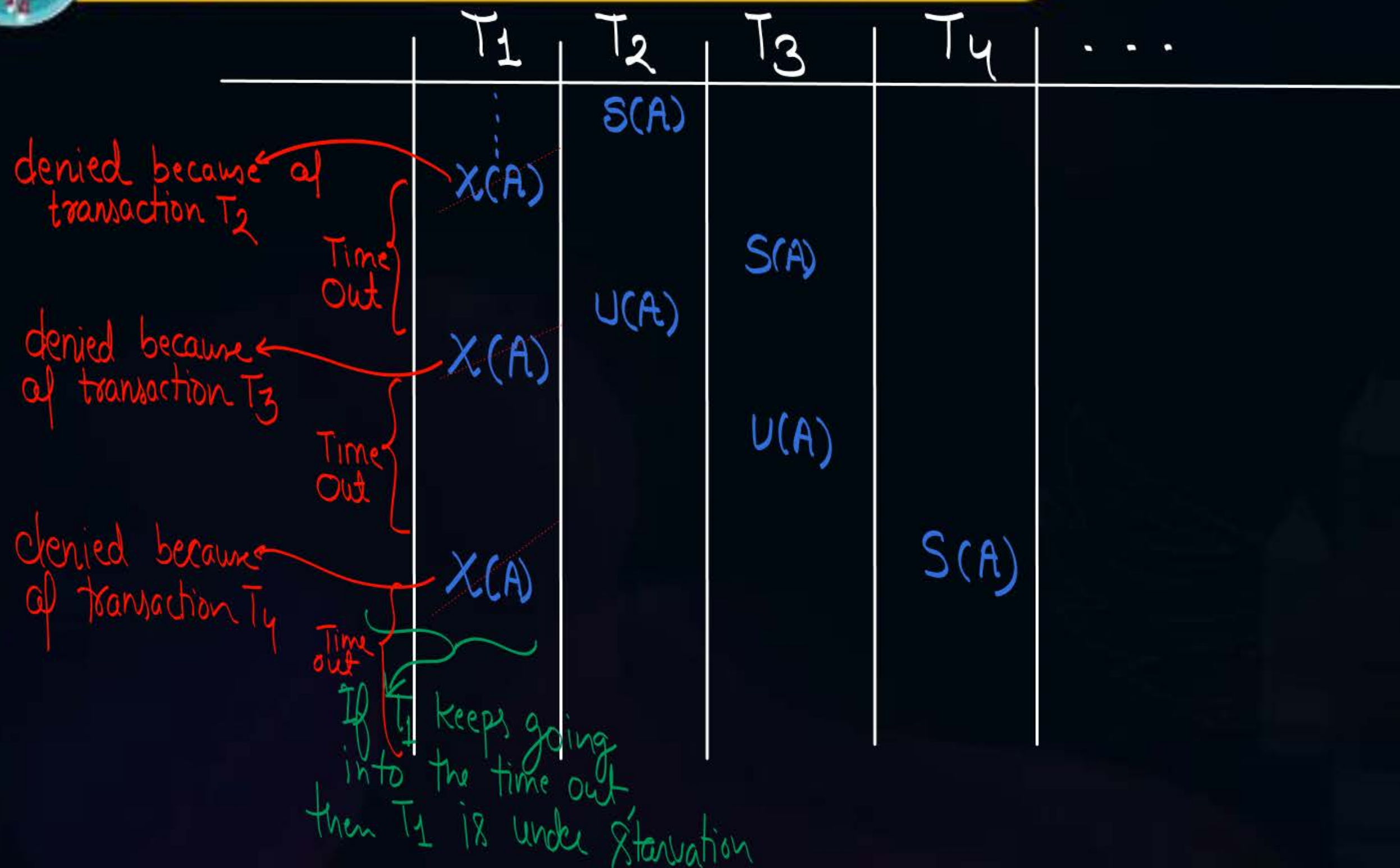
Dependency graph



Cyclic dependency graph
∴ Deadlock is possible



Topic : Starvation with Basic 2PL





Topic : Classification of Schedules based on 2PL

- ① Basic 2PL {Already discussed}
- ② Strict - 2PL
- ③ Conservative 2PL
- ④ Rigorous 2PL

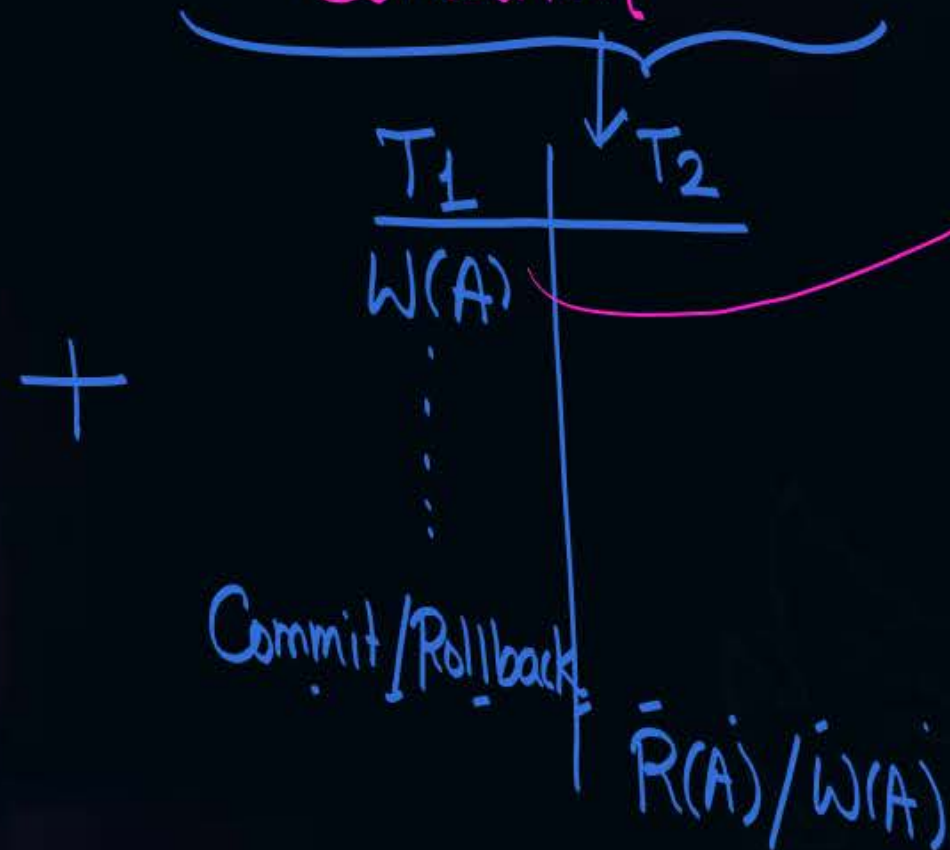


Topic : Strict 2PL

Basic 2PL + Strict recoverability Condition = Strict 2PL

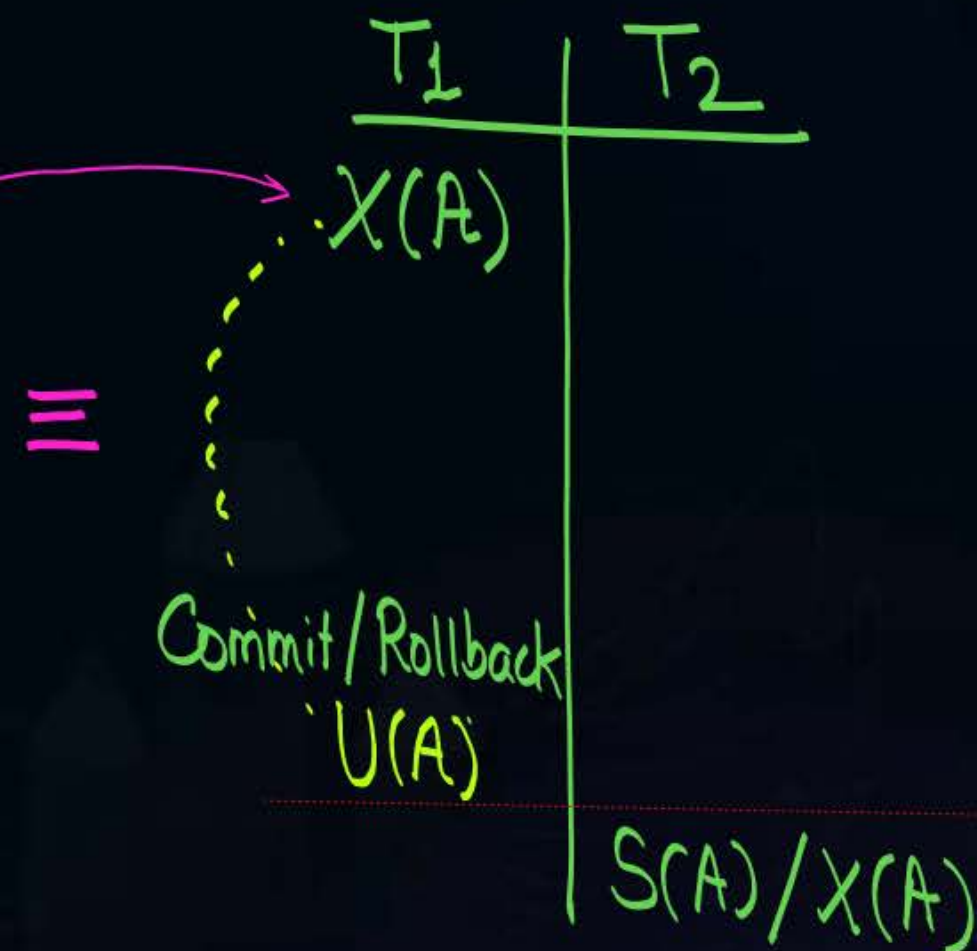
A transaction T is allowed to request for a lock only if it has not performed any unlock opⁿ

it will ensure serializability



+

free from irrecoverability
free from cascading Rollback
free from lost-update





Topic : Strict 2PL



- ★ Strict-2PL is Basic 2PL along with the restriction that if transaction acquired exclusive lock on any data-item, then that Exclusive lock can be unlocked only after the Commit opⁿ of that transaction

Shared locks can be Unlocked at any Point as per 2PL.

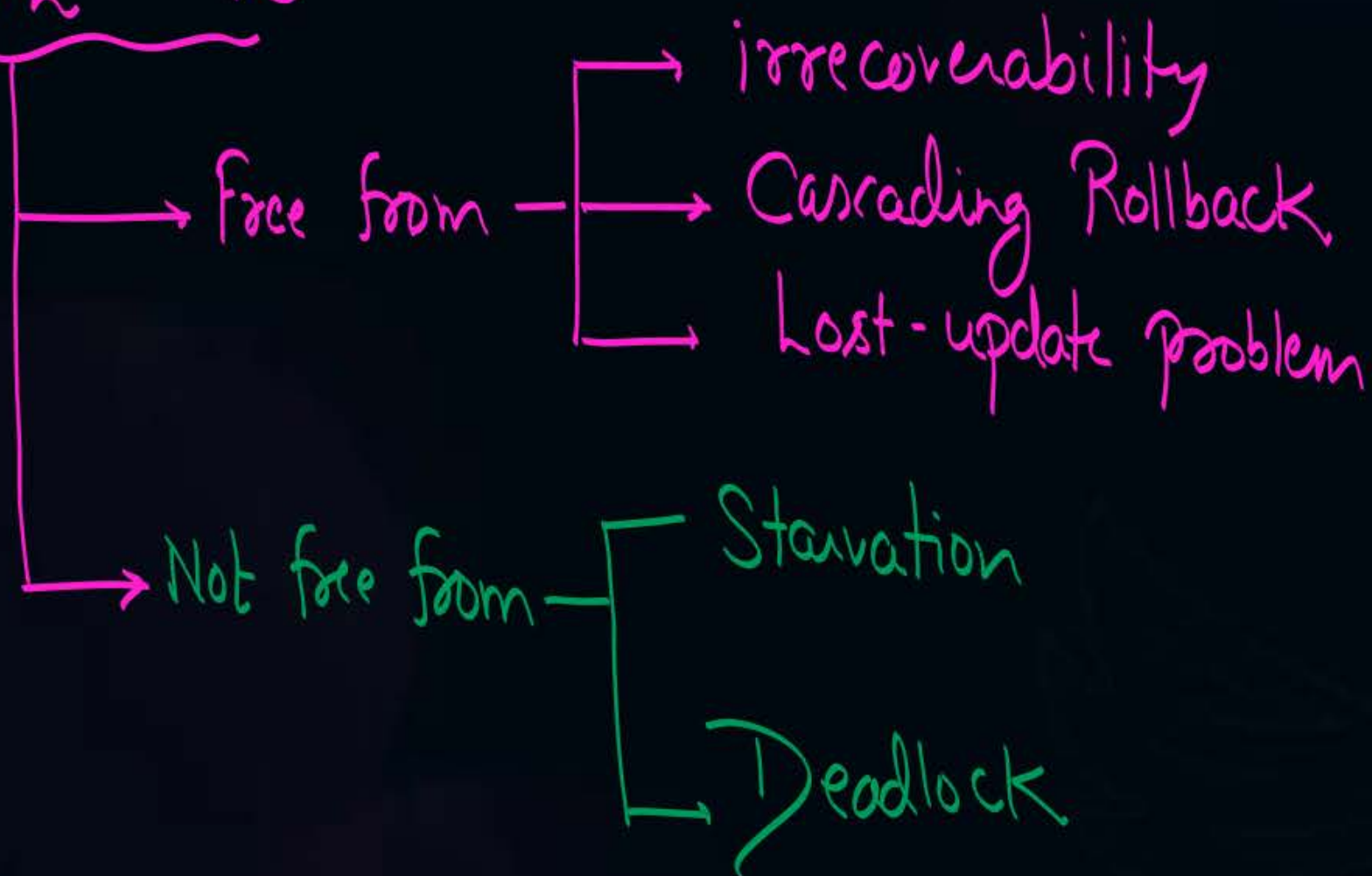
Strict 2PL

T ₁	T ₂
X(A)	
Commit/Rollback U(A)	
	S(A)/X(A)



Topic : Strict 2PL

→ Strict 2PL is



Strict 2PL

T ₁	T ₂
X(A)	
Commit / Rollback U(A)	
	S(A) / X(A)



Topic : Conservative 2PL

it is used to solve the Problem of deadlock



→ In Conservative 2PL we will dis-satisfy Hold-&-wait by Hold-or-wait.

→ In Conservative 2PL the transaction will request for all the locks required for its execution before starting its execution

Necessary Condⁿ for deadlock

① Mutual Exclusion

② No preemption

③ Hold & Wait

④ Circular Wait

ie; transaction will hold all the required locks, then only it will start its execution

→ If all the requested locks are allocated then transaction will start its execution, and we are sure that during its execution it will not depend on any other transaction, because it is already holding all the required locks

(or)
transaction will wait for all the locks to become available at once

→ If at least one of the requested lock is not allocated, then it will release all the allocated locks as well, i.e., it will not hold any lock. And will go into a time out, and after its time out period is over, it will again request for all the locks

We can dis-satisfy Hold & wait by implementing Hold or Wait

∴ No one will depend on this transaction



Topic : Conservative 2PL

* Conservative 2PL only defines the order in which locks will be acquired *i.e., before starting the execution of transaction*, but it does not define the position at which locks will be unlocked. *i.e., Exclusive locks can be released before the commit opⁿ as well.*

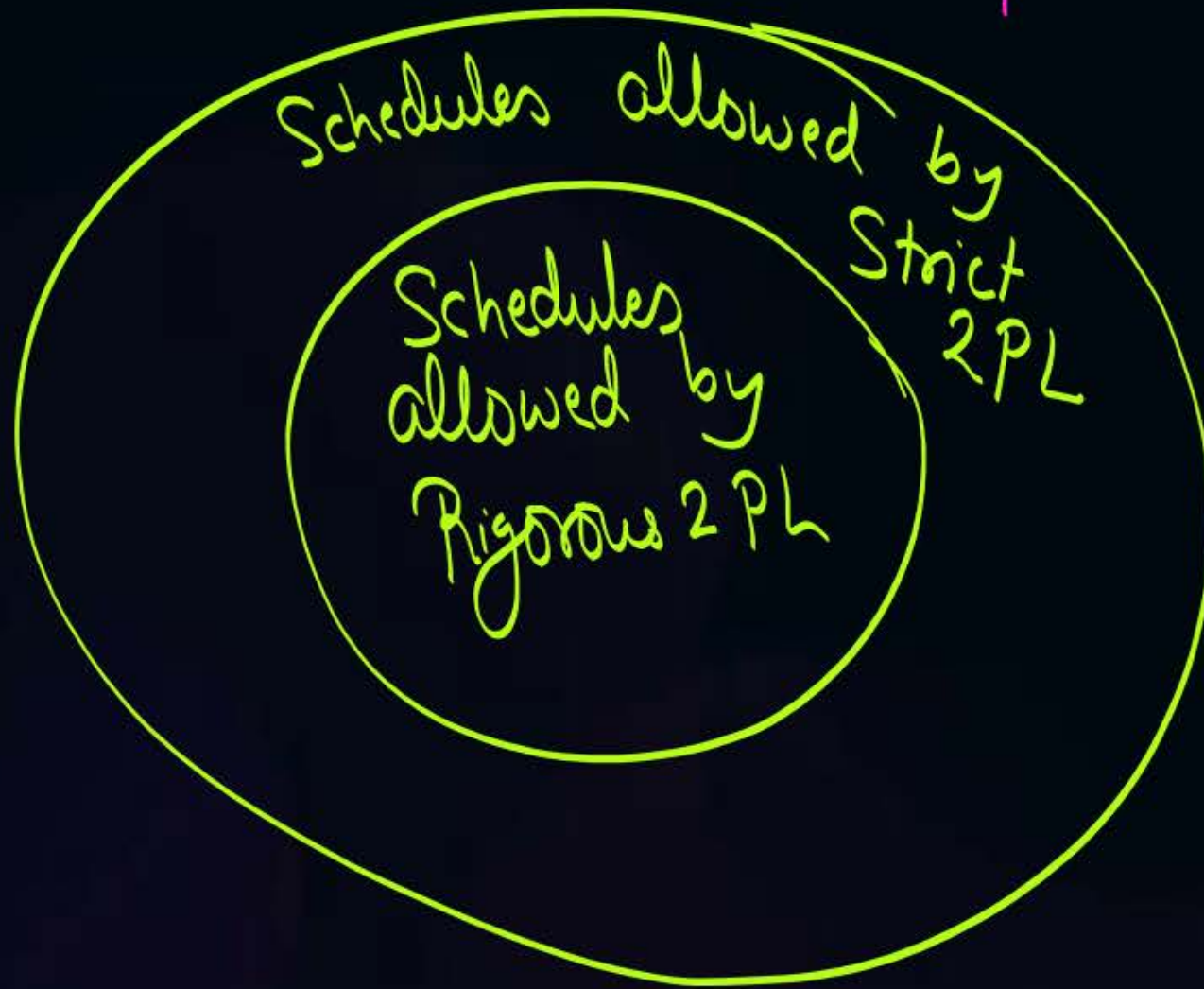
Hence, Conservative 2PL can suffer from irrecoverability, cascading roll back, and lost-update problem.

* Conservative 2PL is free from deadlock, but it may still suffer from starvation as well.



Topic : Rigorous 2PL

- In Rigorous 2PL both Shared (S) as well as Exclusive (X) locks are unlocked after Commit operation





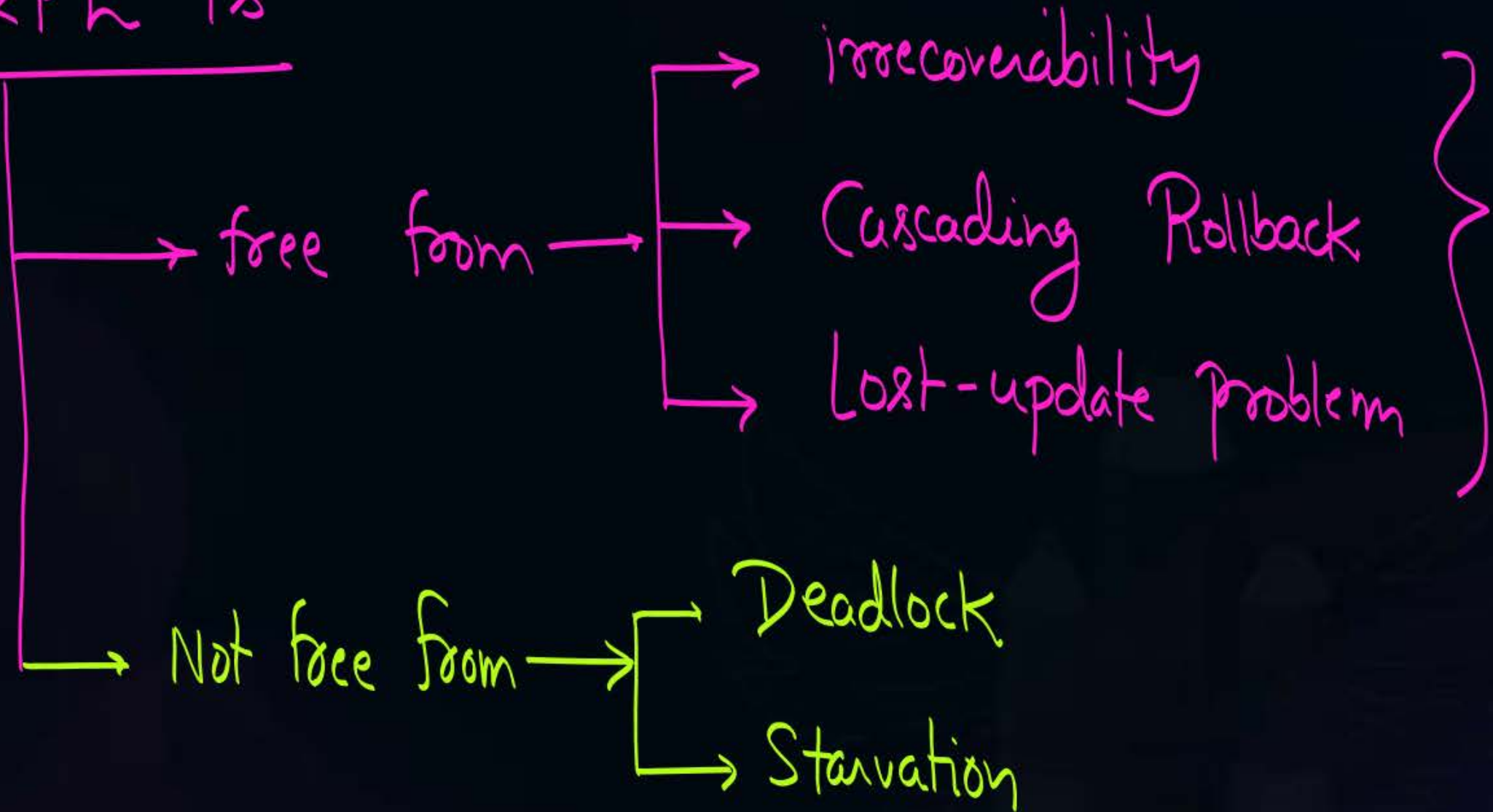
Topic : Rigorous 2PL



Rigorous 2PL is

Implementation
of Rigorous 2PL
is easy

↓
We just need to
Ensure that all
unlock opⁿ are
performed after Commit.





2 mins Summary



✓
Topic

Basic 2PL and lock upgrading/downgrading

✓
Topic

Problems possible with Basic 2PL

✓
Topic

Strict 2PL

✓
Topic

Conservative 2PL

✓
Topic

Rigorous 2PL

T_1	T_2	T_3
$W_1(A)$		
	$W_2(A)$	
		$R(A)$
$R(B)$		
		$W(B)$



THANK - YOU